

SUNSHINE WALLET

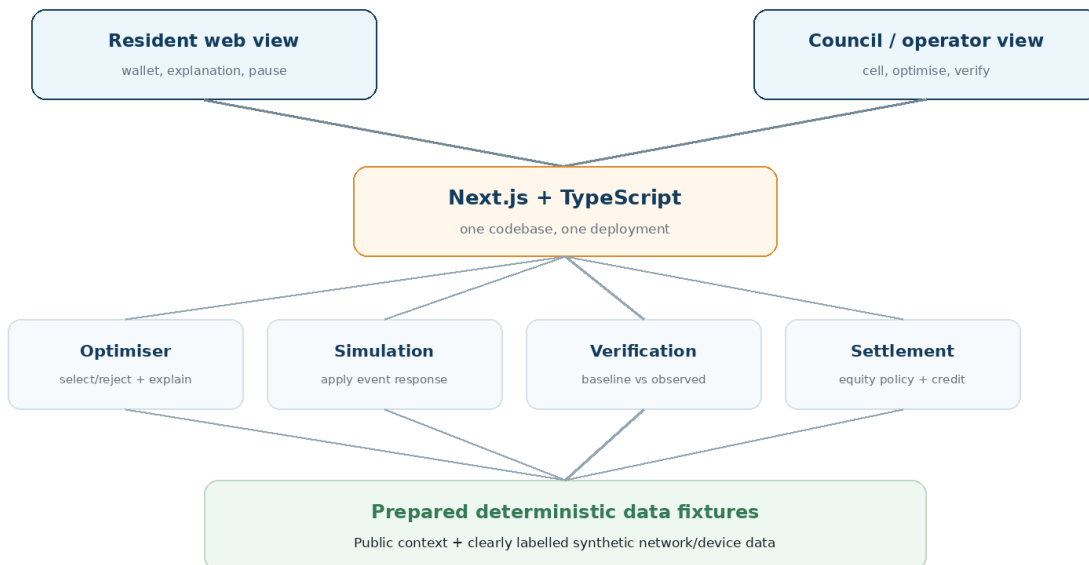
Refined 25-Hour Hackathon Implementation Plan

Team of 3 | One responsive web application | Resident + Council/Operator views

Core build rule

Build one complete, defensible DAPTO-01 event from simulated network need to verified flexibility to an equity-governed resident credit. Everything else is secondary.

25-hour MVP architecture



Prepared from the Sunshine Wallet concept brief and refined for a 25-hour build window.

1. Executive Review: What Needed to Change

The earlier 25-hour plan had the right demo story, but it still carried several implementation risks. This revision removes ambiguity, tightens units and contracts, and makes the project safer for three people working in parallel.

Weakness	Why it matters	Refinement
Scope still slightly too broad	A map, database, multiple areas, wallet, optimiser, verification and settlement could pull the team into parallel half-features.	Only DAPTO-01 must be interactive. Other locations are read-only context cards and can be dropped without damaging the demo.
kW vs kWh was blurred	Power (kW) and energy over an event window (kWh) are not interchangeable. Judges may notice.	Use <code>targetFlexEnergyKwh</code> , <code>maxPowerKw</code> and <code>maxShiftEnergyKwh</code> explicitly. The MVP optimises event energy, not an undefined capacity number.
Confidence treatment was under-specified	Multiplying credits directly by a confidence score would be arbitrary unless justified.	Use confidence as a dispatch/settlement gate. Example: if M&V confidence < 0.70, show the result but do not settle the event.
Settlement value could look invented	A dollar credit without a transparent prototype rule invites "who pays?" questions.	Label the settlement rate as simulated and separate computation from production funding. The real product needs a retailer/aggregator/DNSP program arrangement.
Database was treated as more important than it is	Persistence can consume hours without changing the judged story.	P0 uses deterministic TypeScript/JSON fixtures and in-memory demo state. Supabase is optional after the end-to-end flow works.
Separate resident and council apps could duplicate work	Two apps create routing, styling and deployment overhead.	Build one Next.js application with two experiences: desktop operator dashboard and responsive resident wallet.
Git integration risk was not explicit enough	Three people can easily create merge conflicts in shared layout/type files.	Freeze shared contracts in Hour 1, assign page ownership, use feature branches + small PRs, and keep main always demoable.
No explicit demo reset	A failed or partially completed run can leave state inconsistent during judging.	Add one Reset Demo action that restores the deterministic DAPTO-01 scenario instantly.

Critical correction

The prototype is not proving that electricity can be routed to Dinh. It is proving that a local flexibility event can be selected, simulated, verified and settled under an explicit equity rule. The resident receives program value, not specific electrons.

2. Final Product Decision

2.1 Web application, not native mobile

Build one responsive Next.js web application. The Council/operator experience is desktop-first because it contains charts, resource tables and governance controls. The resident experience is mobile-style and responsive inside the same web application. This gives the visual feel of a consumer wallet without React Native/Expo overhead.

Option	Demo value	25h cost/risk	Decision
Native mobile app	Low	High	Do not build. It adds packaging/device risk without helping the core proof.
Separate Council and resident web apps	Medium	Medium-high	Do not build. Duplicates setup, routing, styling and deployment.
One responsive web app with two role views	Very high	Low-medium	Recommended. One codebase, shared logic, one deployment, easy judge access.

2.2 The only required end-to-end story

```
Resident: Dinh has no rooftop solar
-> Operator opens DAPTO-01
-> 12:00-14:00 simulated local flexibility need is shown
-> Optimiser evaluates 6 resources
-> 4 are selected, 2 are rejected with reasons
-> Event simulation applies the dispatch
-> Verification compares observed response with baseline
-> Equity policy validates settlement
-> Dinh receives a transparent program credit
-> Demo can be reset instantly
```

Core hypothesis

If Sunshine Wallet can turn one credible local flexibility opportunity into a measured technical result and a transparent, equity-constrained credit, the hackathon prototype proves the concept without pretending to operate the real electricity network.

3. Scope: What We Actually Build

3.1 P0 - demo cannot work without this

- One deterministic DAPTO-01 Sunshine Cell scenario.
- One resident persona: Dinh, renter, no rooftop solar, equity-pool eligible.
- 24-hour interval dataset with baseline demand, solar opportunity and an explicitly simulated local network need.
- Six flexible resources with correct cell, availability, comfort and event-energy fields.
- Real optimiser function that selects/rejects resources and explains each decision.
- Real event simulation that produces an observed demand series from the baseline plus actual resource response.
- Real verification function: observed response minus estimated baseline during the event window.
- Confidence threshold logic: low-confidence events are not settled.
- Real settlement calculation using a clearly labelled simulated \$/verified-kWh program rate.
- Equity Floor validation and transparent allocation.
- Resident wallet update showing Dinh's latest credit and explanation.
- Demo reset function.
- One deployable Next.js application and a localhost fallback.

3.2 P1 - high-value only after P0 is stable

- Three read-only area cards (Dapto, Warrawong, Wollongong) to contextualise prioritisation.
- Editable Equity Floor control with policy validation.
- "No event today" state for low confidence / insufficient need.
- Small audit panel showing what is PUBLIC CONTEXT, ESTIMATED or SIMULATED.
- One alternative policy scenario to show that outcomes really change.

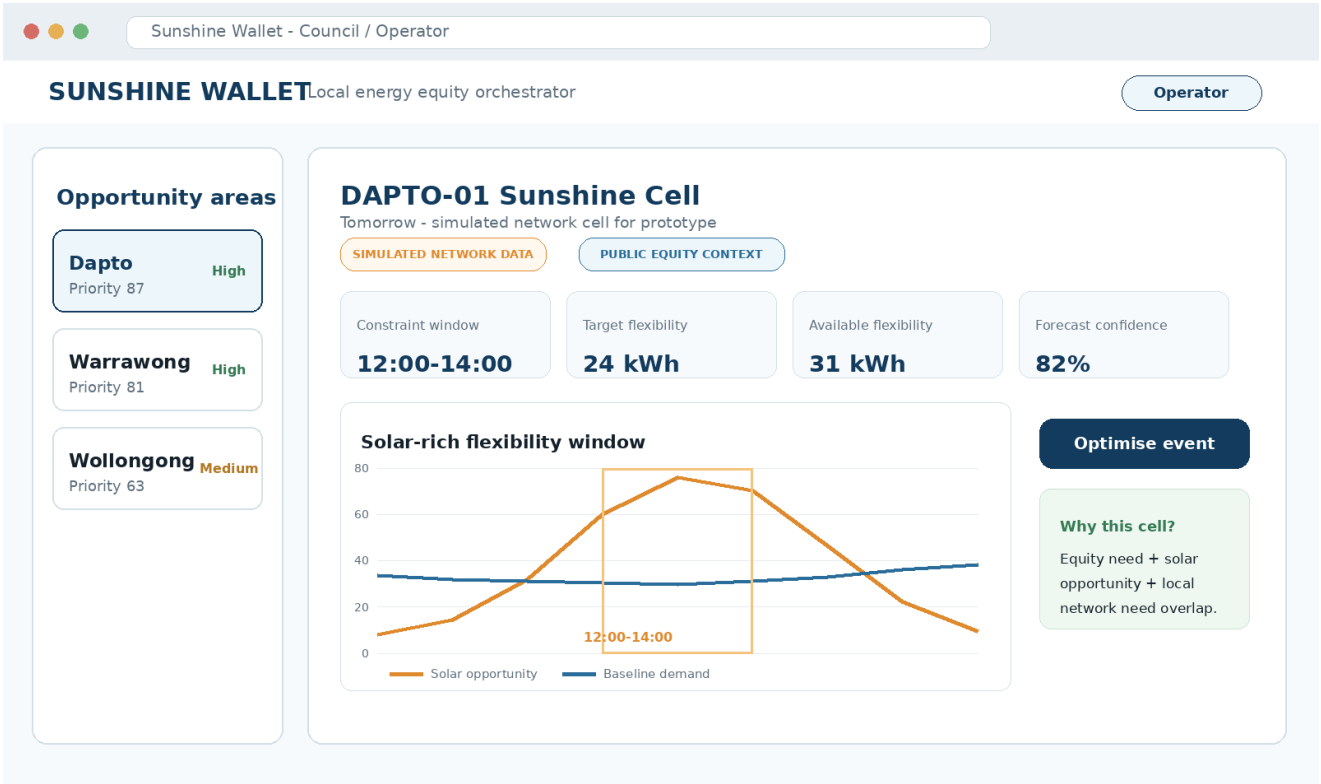
3.3 Do not build in 25 hours

- Native mobile app
- Real authentication / OAuth
- Real smart-meter switching
- Live DNSP voltage/topology integration
- Retailer billing integration
- Real community battery dispatch
- Machine-learning model
- Full GIS map
- Microservices / queues / websockets
- Complex database migrations / admin portal
- Production M&V methodology
- Multiple fully functional Sunshine Cells

4. UI / UX Blueprint

The UI should make the causality obvious. The operator screens prove intelligence and technical discipline; the resident screen proves human value. Do not bury the core event behind menus.

4.1 Council / operator dashboard



Illustrative operator dashboard. DAPTO-01 is the only interactive cell required for the demo.

Judge takeaway: the system is choosing a targeted intervention from an overlap of equity need, solar opportunity and a network need - not merely displaying a hardship map.

4.2 Optimise + governance

Sunshine Wallet - Optimise Event

SUNSHINE WALLET

Local energy equity orchestrator

Operator

Optimise DAPTO-01 event

Target: 24 kWh flexible energy between 12:00 and 14:00

Resource	Type	Max shift	Score	Decision
HW-101	Storage hot water	5 kWh	0.91	SELECTED
HW-102	Storage hot water	4 kWh	0.86	SELECTED
BAT-01	Community battery	10 kWh	0.78	SELECTED
APT-01	Apartment common load	6 kWh	0.76	SELECTED: 5 kWh
EV-04	EV charger	7 kWh	-	REJECTED - unavailable
HW-205	Storage hot water	5 kWh	-	REJECTED - wrong cell

Equity Floor

60%

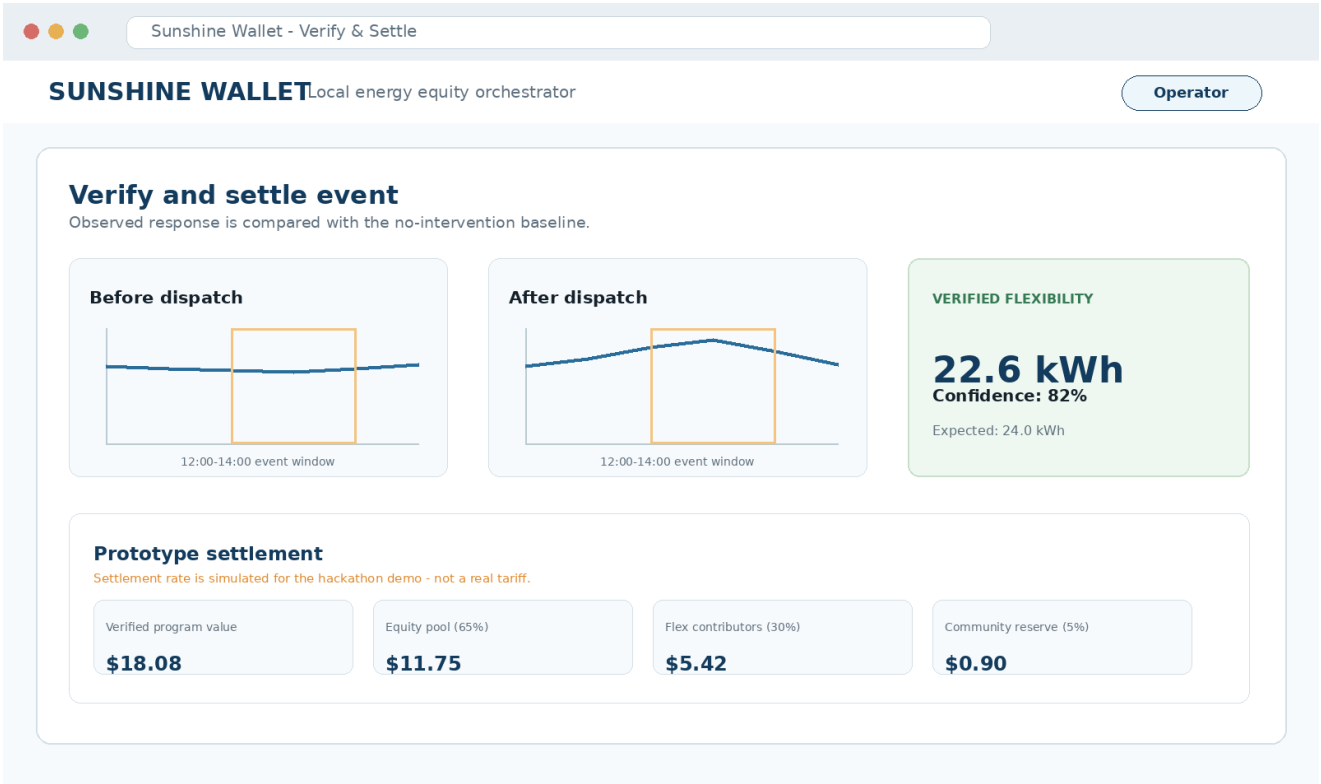
At least 60% of distributable program value must reach equity participants.

Run simulated event

Resource decisions must show why a resource was selected or rejected. This makes the algorithm explainable.

Important: show the event target in kWh, and each resource with max shift energy in kWh for the event window. If you also display maxPowerKw, keep the units separate.

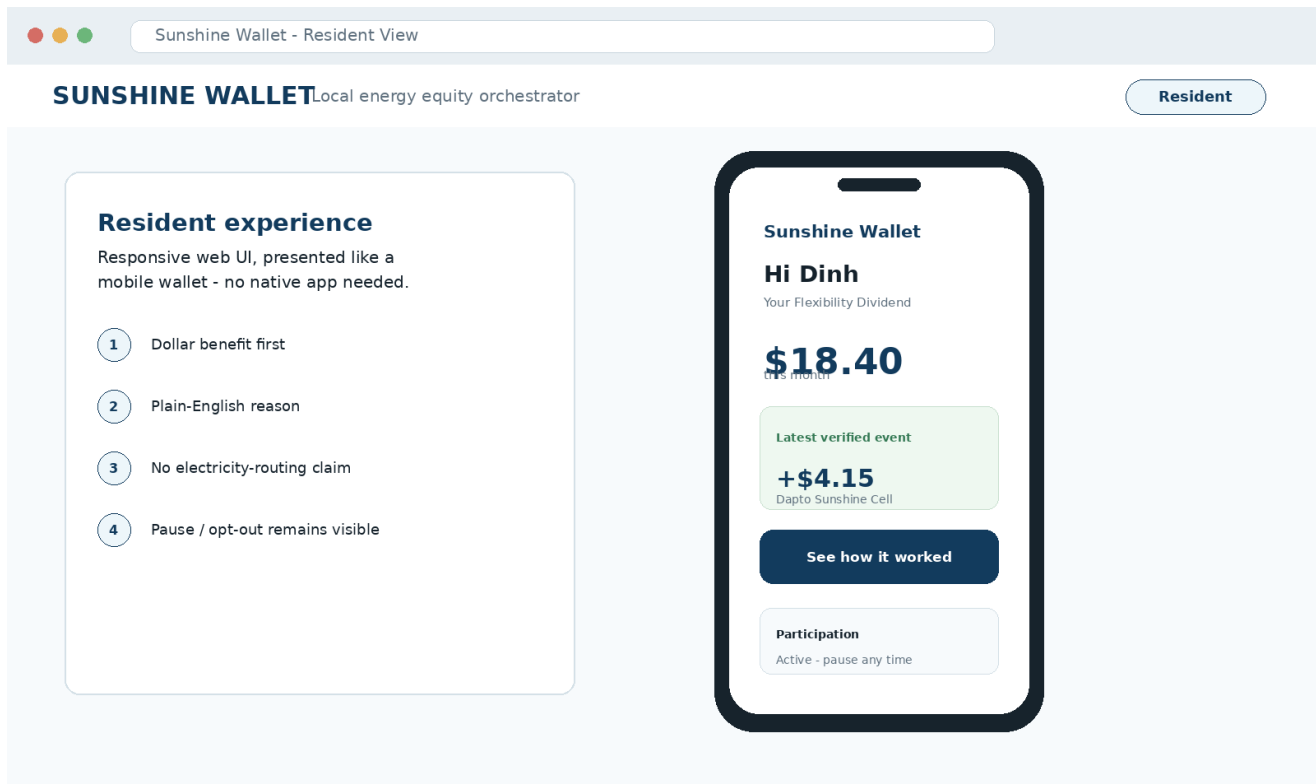
4.3 Verify + settle



Verification and settlement are separate steps. The simulated settlement rate must be labelled clearly.

Judge takeaway: Sunshine Wallet does not reward raw consumption. It compares the observed event response with a counterfactual baseline, applies a confidence rule, then allocates the resulting program value.

4.4 Resident wallet



Resident view is responsive web, presented like a simple mobile wallet. Dollars and explanation come before electrical detail.

The resident should never need to understand DAPR, phases, voltage or dispatch optimisation. The wording should say "verified local flexibility event" or "Flexibility Dividend," not "we sent solar electricity to you."

5. Data Semantics: Fix the Units Before Coding

Why this matters

A technically literate judge can spot unit errors immediately. The MVP should distinguish power (kW), energy over the event window (kWh), confidence (0-1) and currency (\$).

Field	Unit	Meaning
targetFlexEnergyKwh	kWh	Energy the event wants to shift/absorb during the complete window.
maxPowerKw	kW	Maximum instantaneous controllable power of a resource.
maxShiftEnergyKwh	kWh	Maximum energy that resource can shift during this event.
expectedContributionKwh	kWh	Planned contribution after optimiser selection.
actualContributionKwh	kWh	Simulated realised contribution.
verifiedFlexibilityKwh	kWh	Observed event response minus estimated baseline.
confidence	0-1	Confidence in the event/verification estimate; use for gating, not arbitrary monetary multiplication.
settlementRateAudPerKwh	AUD/kWh	Prototype-only value rate, clearly marked simulated.

5.1 Deterministic hero scenario

Item	Value	Design note
Cell	DAPTO-01	Synthetic/approximate network zone for demo.
Event window	12:00-14:00	2-hour local solar-rich window.
Target flexible energy	24.0 kWh	Event objective.
Forecast/M&V confidence	0.82	Above prototype minimum 0.70.
Selected expected contribution	24.0 kWh	5 + 4 + 10 + 5.
Observed verified flexibility	22.6 kWh	Underperformance is intentional and more believable than perfect delivery.

Item	Value	Design note
Prototype settlement rate	\$0.80/kWh	\$18.08 total program value; simulated, not a real market tariff.
Equity Floor	60%	Policy minimum.
Actual equity pool share	65%	\$11.75, therefore valid.
Dinh credit	\$4.15	Example weighted share of the equity pool.

6. Minimum Data Model

For P0, these are TypeScript types/JSON objects rather than database tables. If Supabase is added later, the same model can be normalised into tables. Do not let persistence block the vertical slice.

```

type Household = {
  id: string;
  displayName: string;
  sunshineCellId: string;
  householdType: "renter" | "owner" | "apartment";
  rooftopSolar: boolean;
  equityEligible: boolean;
  equityNeedWeight: number; // prototype policy weight, e.g. 1-3
  consent: "active" | "paused";
  participationMode: "flexibility" | "equity_pool";
};

type SunshineCell = {
  id: string;
  displayName: string;
  suburb: string;
  equityScore: number; // 0-100, public-context prototype score
  solarOpportunityScore: number; // 0-100
  networkOpportunityScore: number; // 0-100, synthetic for hero cell
  confidence: number; // 0-1
  dataClassification: "public" | "estimated" | "simulated";
};

type FlexResource = {
  id: string;
  sunshineCellId: string;
  householdId?: string;
  type: "hot_water" | "community_battery" | "ev" | "apartment_common_load";
  maxPowerKw: number;
  maxShiftEnergyKwh: number;
  available: boolean;
  comfortSafe: boolean;
  controllabilityScore: number; // 0-1
  networkEffectivenessScore: number; // 0-1
  equityNeedScore: number; // 0-1
  rotationFairnessScore: number; // 0-1
};

type FlexEvent = {
  id: string;
  sunshineCellId: string;
  startIso: string;

```

```

    endIso: string;
    targetFlexEnergyKwh: number;
    forecastConfidence: number;
    minConfidenceForDispatch: number;
    status: "forecast" | "optimised" | "simulated" | "verified" | "settled" | "no_event";
  };

type DispatchDecision = {
  eventId: string;
  resourceId: string;
  selected: boolean;
  score?: number;
  expectedContributionKwh: number;
  reason: string;
};

type VerificationResult = {
  eventId: string;
  baselineEnergyKwh: number;
  observedEnergyKwh: number;
  verifiedFlexibilityKwh: number;
  confidence: number;
  settlementEligible: boolean;
};

type CreditLedgerEntry = {
  id: string;
  eventId: string;
  householdId: string;
  amountAud: number;
  channel: "equity_pool" | "flex_contributor";
  reason: string;
  policyVersion: string;
};

```

7. Recommended JSON / API Contract

Preferred 25-hour implementation

Use a single server-side demo service (or server actions) rather than a large REST surface. Still define stable request/response contracts so Person 2 and Person 3 can build against mock fixtures while Person 1 implements the engine.

7.1 GET /api/demo/state

```
{
  "scenarioId": "dapto-demo-v1",
  "household": {
    "id": "hh-dinh",
    "displayName": "Dinh",
    "sunshineCellId": "DAPTO-01",
    "householdType": "renter",
    "rooftopSolar": false,
    "equityEligible": true,
    "equityNeedWeight": 2,
    "consent": "active",
    "participationMode": "equity_pool"
  },
  "cell": {
    "id": "DAPTO-01",
    "displayName": "Dapto Sunshine Cell",
    "suburb": "Dapto",
    "equityScore": 82,
    "solarOpportunityScore": 86,
    "networkOpportunityScore": 79,
    "confidence": 0.82,
    "dataClassification": "simulated"
  },
  "event": {
    "id": "evt-001",
    "startIso": "2026-08-23T12:00:00+10:00",
    "endIso": "2026-08-23T14:00:00+10:00",
    "targetFlexEnergyKwh": 24.0,
    "forecastConfidence": 0.82,
    "minConfidenceForDispatch": 0.70,
    "status": "forecast"
  },
  "wallet": {
    "monthToDateAud": 11.60,
    "latestCreditAud": 0
  }
}
```

7.2 POST /api/demo/optimise

```
Request
{
  "eventId": "evt-001",
  "equityFloor": 0.60
}
```

```

Response
{
  "eventId": "evt-001",
  "targetFlexEnergyKwh": 24.0,
  "expectedSelectedEnergyKwh": 24.0,
  "decisions": [
    {"resourceId": "HW-101", "selected": true, "score": 0.91, "expectedContributionKwh": 5.0, "reason": "high network effectiveness; available; comfort safe"},
    {"resourceId": "HW-102", "selected": true, "score": 0.86, "expectedContributionKwh": 4.0, "reason": "available; equity priority; correct cell"},
    {"resourceId": "BAT-01", "selected": true, "score": 0.78, "expectedContributionKwh": 10.0, "reason": "correct cell; controllable"},
    {"resourceId": "APT-01", "selected": true, "score": 0.76, "expectedContributionKwh": 5.0, "reason": "partial dispatch reaches event target"},
    {"resourceId": "EV-04", "selected": false, "expectedContributionKwh": 0, "reason": "unavailable"},
    {"resourceId": "HW-205", "selected": false, "expectedContributionKwh": 0, "reason": "wrong Sunshine Cell"}
  ]
}

```

7.3 POST /api/demo/run-and-verify

```

Response
{
  "eventId": "evt-001",
  "expectedFlexibilityKwh": 24.0,
  "actualResourceResponseKwh": 22.6,
  "verification": {
    "baselineEnergyKwh": 78.4,
    "observedEnergyKwh": 101.0,
    "verifiedFlexibilityKwh": 22.6,
    "confidence": 0.82,
    "settlementEligible": true
  }
}

```

7.4 POST /api/demo/settle

```

Request
{
  "eventId": "evt-001",
  "equityFloor": 0.60,
  "equityPoolShare": 0.65,
  "contributorShare": 0.30,
  "communityReserveShare": 0.05,
  "settlementRateAudPerKwh": 0.80
}

Response
{
  "verifiedFlexibilityKwh": 22.6,
  "totalProgramValueAud": 18.08,
  "policyValid": true,
  "allocation": {
    "equityPoolAud": 11.75,
    "flexContributorsAud": 5.42,
    "communityReserveAud": 0.90
  },
  "credits": [
    {"householdId": "hh-dinh", "amountAud": 4.15, "channel": "equity_pool", "reason": "eligible household share under policy v1"}
  ],
  "walletAfter": {

```

```
"monthToDateAud": 15.75,  
"latestCreditAud": 4.15  
}  
}
```

7.5 POST /api/demo/reset

```
Response  
{  
  "scenarioId": "dapto-demo-v1",  
  "status": "reset",  
  "message": "Demo state restored to deterministic starting point."  
}
```

8. Core Business Logic

8.1 Resource eligibility first, scoring second

Hard constraints must be checked before ranking. This prevents a high-scoring but unavailable or wrong-cell device from being selected.

```
for each resource:
  if resource.sunshineCellId != event.sunshineCellId:
    reject("wrong Sunshine Cell")
  else if !resource.available:
    reject("unavailable")
  else if !resource.comfortSafe:
    reject("comfort constraint")
  else:
    score =
      0.35 * networkEffectivenessScore +
      0.30 * equityNeedScore +
      0.20 * controllabilityScore +
      0.15 * rotationFairnessScore
    candidate(score)

sort candidates descending by score
select contributions until targetFlexEnergyKwh is met
allow final resource to be partially dispatched
```

8.2 Event simulation

Do not pretend to run a power-flow model. The hackathon engine applies deterministic resource response factors to the baseline interval series. This is a behavioural simulation, clearly labelled as such.

```
actualContributionKwh = expectedContributionKwh * responseFactor

Example response factors:
HW-101 0.95
HW-102 0.90
BAT-01 0.98
APT-01 0.82

sum(actualContributionKwh) = 22.6 kWh
observedIntervalSeries = baselineIntervalSeries + allocated actual response
```

8.3 Verification

```
verifiedFlexibilityKwh =
  observedEnergyDuringEventKwh - estimatedBaselineEnergyDuringEventKwh

settlementEligible =
  verificationConfidence >= minimumConfidence
```

```
AND verifiedFlexibilityKwh > 0
```

The confidence score is displayed and used as a minimum-quality gate. Do not multiply the dollar value by confidence unless you explicitly present that as a prototype conservative rule; it is not necessary for this MVP.

8.4 Settlement + Equity Floor

```
totalProgramValueAud = verifiedFlexibilityKwh * settlementRateAudPerKwh
```

```
assert equityPoolShare + contributorShare + reserveShare == 1.0
assert equityPoolShare >= equityFloor
```

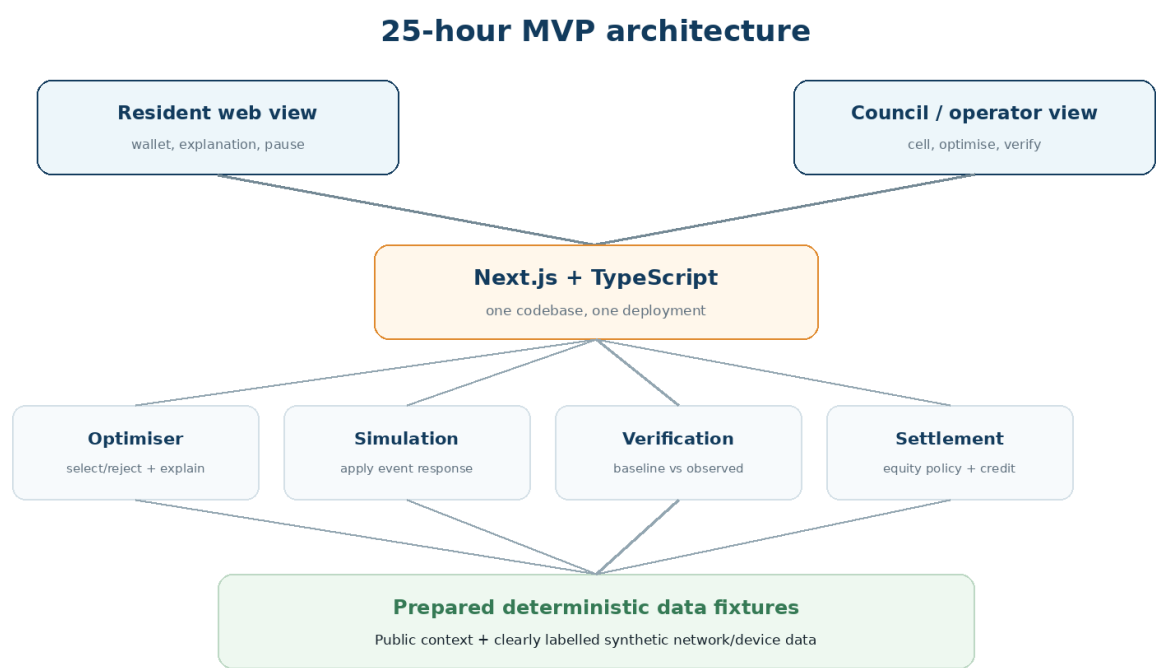
```
if either assertion fails:
    reject policy configuration
else:
    equityPoolAud = totalProgramValueAud * equityPoolShare
    contributorAud = totalProgramValueAud * contributorShare
    reserveAud = totalProgramValueAud * reserveShare
```

```
Dinh share example:
credit = equityPoolAud * Dinh.needWeight / sum(eligibleNeedWeights)
```

Important framing

The \$0.80/kWh rate is a simulated demo parameter. The production question "who pays?" remains a partner/program design requirement. The prototype proves the ledger and governance logic, not a tariff.

9. Simplest Architecture That Supports the Demo



Component	Responsibility	25h status	Why
Next.js + TypeScript	All UI, route handlers/server actions and shared types.	Must implement	One repo, one deployment, minimal integration overhead.
Tailwind + component primitives	Layout, cards, buttons, tables, responsive resident UI.	Must implement	Fast consistency; do not hand-design a component system.
Recharts	24-hour baseline/observed charts.	Must implement	High visual payoff for low effort.
Demo engine in lib/	optimise, simulate, verify, settle, reset.	Must implement	This is the actual technical proof.
Typed fixture data	DAPTO-01, Dinh, resources, intervals.	Must implement	Deterministic and offline-safe.
Supabase/PostgreSQL	Persistence.	Optional after Hour 12	No judge value if core flow is not stable.
External live APIs	Weather, retailer, DNSP etc.	Do not implement	Fragile and not required for proof.

10. Exact Clickable Flow

1. Start on Dinh's Wallet. Show that he rents, has no rooftop solar and currently has an \$11.60 month-to-date program benefit.
2. Click "See how today's event works" and switch to the operator DAPTO-01 dashboard.
3. Show the 12:00-14:00 event, 24 kWh target and 82% confidence. Point out the SIMULATED NETWORK DATA badge.
4. Click "Optimise event." Six resources are evaluated; four selected; two rejected with reasons.
5. Show the 60% Equity Floor. If time permits, briefly test an invalid policy share and show the UI refusing it.
6. Click "Run simulated event." The observed demand curve changes during the event window.
7. Show 22.6 kWh verified flexibility and 82% verification confidence.
8. Click "Settle." The system calculates \$18.08 simulated program value and validates 65% equity allocation against the 60% floor.
9. Return to Dinh. Wallet increases from \$11.60 to \$15.75 with a +\$4.15 latest credit.
10. Use "Reset Demo" before the next judging run.

11. 25-Hour Build Schedule

Hours	Owner	Work	Exit condition
0-1	All	Freeze scope, hero scenario, units, routes, shared types, Git ownership. No feature coding until contracts agree.	README contains one-line demo story and ownership matrix.
1-2	Parallel	Scaffold Next.js; create feature branches; create fixtures/types; make operator and resident shells.	All three can work independently against shared mock objects.
2-5	Parallel	P1 core engine; P2 operator dashboard; P3 resident wallet + visual system + pitch outline.	By Hour 5, engine prints deterministic expected results.
5-8	Integration	Connect optimise -> simulate -> verify -> settle into operator flow.	Emergency vertical slice works, even if ugly.
8-11	Integration	Connect resident wallet update, reset demo, rejection reasons, data badges.	Complete story works end-to-end.
11-13	Polish	Charts, responsive layout, policy validation, error/no-event state if cheap.	Submission-quality core UI.
13	ALL	FEATURE FREEZE.	No new meaningful features.
13-16	Reliability	Deploy Vercel; local fallback; reset tests; refresh tests; seed consistency.	Demo survives refresh and poor internet.
16-18	Presentation	README, architecture image, real-vs-simulated explanation, backup screenshots/video.	Technical story and fallback assets ready.
18	ALL	CODE FREEZE.	Only critical bug fixes after this point.
18-21	Rehearsal	3-5 minute pitch, judge Q&A, handoff between presenter/operator.	At least two clean timed runs.
21-23	Buffer	Critical bug fixes only, accessibility/legibility, submission packaging.	No architectural changes.
23-25	Final	Final reset, offline check, tabs ready, notifications off, final rehearsal.	Stable demo and rested team.

Non-negotiable milestones

Hour 5: engine works. Hour 8: emergency vertical slice works. Hour 11: complete demo works. Hour 13: feature freeze. Hour 18: code freeze.

12. Team of 3: Ownership and Parallel Work

Person	Primary ownership	First task	Conflict rule
Person 1 - Core / integration	lib/demo-engine, app/api/demo, shared types	Implement optimiser, simulation, verification, settlement, reset. Own deployment plumbing.	Avoid editing operator/resident page markup unless requested.
Person 2 - Operator UI	app/operator, components/operator, charts	Build DAPTO-01 dashboard, resources table, governance, verify/settle view.	Use shared types/fixtures; do not redefine interfaces locally.
Person 3 - Resident / UX / pitch	app/wallet, components/resident, global visual polish, README/pitch	Build Dinh wallet, navigation/role switch, responsive polish, demo script, judge Q&A.	Avoid changing engine files. Coordinate shared styling before broad edits.

12.1 First two hours checklist

- Create GitHub repository and add all 3 members.
- Create main and dev branches; protect main if practical.
- Create shared types in src/types/demo.ts before parallel UI work.
- Create src/data/demoScenario.ts with DAPTO-01 deterministic fixture.
- Create routes /wallet and /operator.
- Create lib/demo-engine placeholders with function signatures.
- Agree exact output values for the hero scenario.
- Add .env.example even if no environment variables are currently required.
- Add npm scripts: dev, build, lint, test, demo:check.
- Open three GitHub issues and assign one to each person.
- Push a deployable skeleton to dev and verify Vercel once early.

13. Failure / Fallback Plan

Risk	Primary	Fallback
Vercel or internet fails	Hosted Next.js app	Run locally with npm run dev; all data/assets bundled locally.
Supabase setup consumes time	Optional persistence	Remove it. Fixture/in-memory state is P0.
Map becomes difficult	Small clickable map/GeoJSON	Use three area cards; DAPTO-01 remains the only interactive cell.
Chart library issue	Recharts	Use a simple SVG/CSS mini-chart or numeric before/after cards.
Engine integration slips	API/server actions	Import engine functions directly into server component/action for demo.
Policy slider causes bugs	Interactive slider	Freeze Equity Floor at 60%; keep policy validation in engine.
Wallet update persistence fails	Persistent DB	Pass settlement result through app state / server demo state; reset between runs.
Live data/API unavailable	External API	Use cached/prepared public context and synthetic network fixture.
Demo state corrupted	Interactive state	Reset Demo restores exact fixture from source object.
One teammate blocked	Shared dependency	Mock the frozen JSON contract and continue independently.

14. What Is Real vs Simulated

Category	What belongs here
Real code	Resource eligibility/ranking, partial selection to event target, state transitions, verification arithmetic, policy validation, settlement arithmetic, wallet update.
Prepared / public context	Area-level equity/solar context where you have a defensible source. Label the source and granularity.
Simulated	DAPTO-01 electrical topology/constraint, controllable resources, response factors, event telemetry, settlement rate, retailer/DNSP responses.
Not claimed	Direct routing of electricity to Dinh, actual bill credit, real DNSP control, real tariff value, regulatory-grade M&V.

Data honesty rule

Every synthetic electrical or market value should carry a SIMULATED badge in the operator experience. This is a credibility feature, not an apology.

15. Tiny Test Suite with High Value

Test	Expected	Why
optimise rejects wrong cell	HW-205 is never selected.	Prevents the most damaging topology mistake.
optimise rejects unavailable	EV-04 is never selected.	Proves hard constraints beat scores.
partial final dispatch	APT-01 contributes only what is needed to reach 24 kWh.	Avoids overshoot and shows deliberate optimisation.
low confidence -> no settlement	confidence 0.60 returns settlementEligible=false.	Supports "No event / no settlement" credibility.
invalid policy blocked	equityPoolShare 0.50 with floor 0.60 fails.	Makes fairness executable.
allocation sums to total	equity + contributor + reserve == total within rounding tolerance.	Avoids embarrassing arithmetic discrepancy.
reset restores fixture	wallet and event return to initial values.	Essential for repeated judging.

16. Final Project Folder Structure

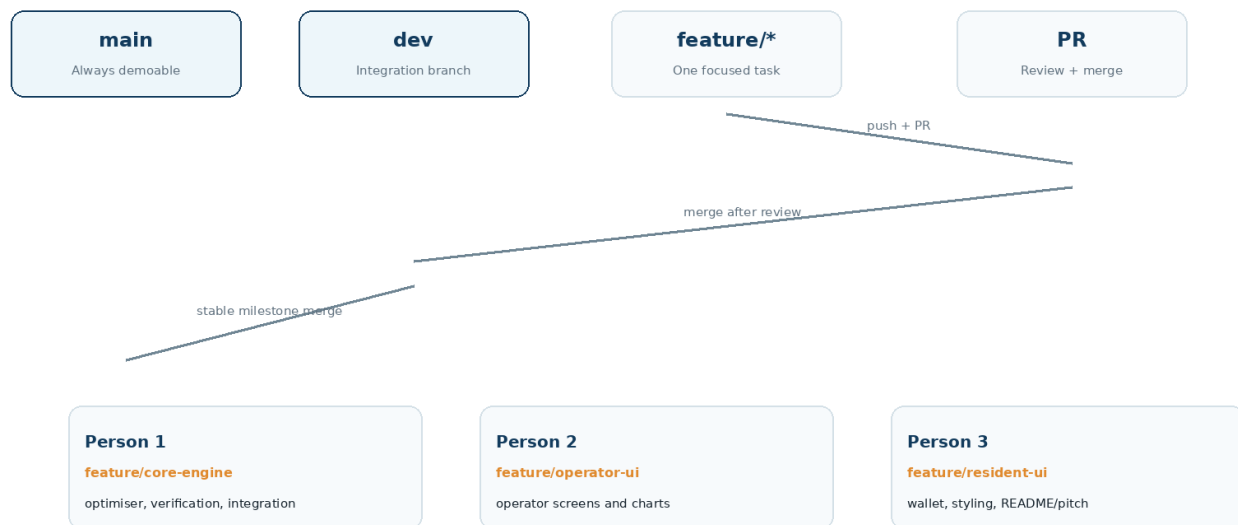
```
sunshine-wallet/  
├── src/  
│   ├── app/  
│   │   ├── page.tsx          # simple demo entry / role choice  
│   │   ├── wallet/  
│   │   │   └── page.tsx      # Dinh resident experience  
│   │   ├── operator/  
│   │   │   └── page.tsx      # DAPTO-01 operator journey  
│   │   └── api/  
│   │       ├── demo/  
│   │       │   ├── state/route.ts  
│   │       │   ├── optimise/route.ts  
│   │       │   ├── run-and-verify/route.ts  
│   │       │   ├── settle/route.ts  
│   │       │   └── reset/route.ts  
│   ├── components/  
│   │   ├── shared/  
│   │   │   ├── DataBadge.tsx  
│   │   │   ├── MetricCard.tsx  
│   │   │   └── DemoNav.tsx  
│   │   ├── operator/  
│   │   │   ├── EnergyChart.tsx  
│   │   │   ├── ResourceDecisionTable.tsx  
│   │   │   ├── EquityPolicyPanel.tsx  
│   │   │   └── VerificationPanel.tsx  
│   │   └── resident/  
│   │       ├── WalletBalance.tsx  
│   │       └── LatestCreditCard.tsx  
│   ├── lib/  
│   │   ├── demo-engine/  
│   │   │   ├── optimise.ts  
│   │   │   ├── simulate.ts  
│   │   │   ├── verify.ts  
│   │   │   ├── settle.ts  
│   │   │   └── index.ts  
│   │   ├── demo-state.ts      # in-memory/demo state + reset  
│   │   └── money.ts           # rounding helpers  
│   ├── data/  
│   │   └── demoScenario.ts    # single source of truth fixture  
│   ├── types/  
│   │   └── demo.ts            # shared API/domain types  
│   ├── tests/  
│   │   ├── optimise.test.ts  
│   │   ├── verification.test.ts  
│   │   ├── settlement.test.ts  
│   │   └── reset.test.ts  
│   ├── public/  
│   │   └── assets/            # local logos / static visuals only  
│   ├── docs/  
│   │   ├── demo-script.md  
│   │   ├── real-vs-simulated.md  
│   │   └── architecture.md  
│   ├── .env.example  
│   ├── .gitignore  
│   ├── README.md  
│   ├── package.json  
│   └── tsconfig.json
```

16.1 Folder ownership to reduce merge conflicts

Owner	Primary files
Person 1	src/lib/**, src/app/api/**, src/types/demo.ts, tests/**
Person 2	src/app/operator/**, src/components/operator/**
Person 3	src/app/wallet/**, src/components/resident/**, docs/**, most README work
Shared / coordinate before editing	src/app/layout.tsx, src/app/globals.css, src/components/shared/**, package.json

17. GitHub Setup and Commit Workflow

GitHub workflow for 3 people



17.1 Branch strategy

For a 25-hour hackathon, a lightweight dev + feature branch model is safer than everyone pushing directly to main.

```
main
- always demoable
- deployed production/demo branch
- only receives stable milestone merges

dev
- integration branch
```


- receives reviewed feature PRs
- Vercel preview / staging if available

```
feature/core-engine  
feature/operator-ui  
feature/resident-ui  
feature/<small-task-name>
```

17.2 Initial setup commands

```
git clone <repo-url>  
cd sunshine-wallet  
npm install  
  
git checkout -b dev  
git push -u origin dev  
  
# Each person branches from latest dev  
git checkout dev  
git pull origin dev  
git checkout -b feature/core-engine  
# or feature/operator-ui / feature/resident-ui
```

17.3 How everyone should commit

- Commit small, coherent units every 30-90 minutes; do not wait six hours and push one giant commit.
- Before starting a new chunk: git checkout dev && git pull origin dev, then branch or rebase your feature branch.
- Never commit directly to main during active development.
- Prefer one PR per coherent feature: "core optimiser", "operator resource table", "resident wallet" - not one PR per tiny CSS tweak.
- At least one teammate glances at each PR before merging to dev. For urgent tiny fixes, pair-review verbally and merge quickly.
- After a feature PR merges to dev, the owner pulls dev before continuing.
- Only merge dev -> main when the demo path has been manually run from start to finish.
- At feature freeze, stop creating broad branches. Use small fix/* branches only.
- At code freeze, main is the source of truth for judging; no unreviewed late commits.

17.4 Commit message convention

```
feat(engine): select eligible resources to event target  
feat(operator): add resource decision table  
feat(wallet): show latest equity-pool credit  
fix(settlement): enforce 60 percent equity floor  
fix(demo): restore wallet balance on reset  
test(engine): reject wrong-cell resources  
docs(demo): clarify simulated settlement rate
```

17.5 Pull request checklist

- Branch is updated with dev.
- npm run lint passes.
- Relevant tests pass.
- No unrelated files are changed.
- UI screenshot added for visual changes if convenient.
- Demo path still works for touched screens.
- Any simulated value remains clearly labelled.
- PR description states what changed and how to test it in 2-4 bullets.

17.6 Merge conflict prevention

- Do not have all three people edit globals.css or layout.tsx independently. Assign one owner for shared styling changes.
- Freeze src/types/demo.ts after Hour 2 unless Person 1 coordinates a contract change with both UI developers.
- Use component boundaries so Person 2 and Person 3 rarely touch the same files.
- If a contract must change, update the mock fixture and TypeScript type in the same commit, announce it immediately, then both UI owners pull dev.
- Avoid broad automated formatting halfway through the hackathon; it creates meaningless conflicts.

18. README Structure for Submission

```
# Sunshine Wallet

## One-line pitch
We turn a verified local flexibility event into transparent program value,
with an Equity Floor so households locked out of rooftop solar share the benefit.

## What the prototype demonstrates
1. DAPTO-01 event selection
2. Resource eligibility + optimisation
3. Simulated dispatch
4. Counterfactual verification
5. Equity policy validation
6. Resident wallet credit

## What is real vs simulated
- Real: optimiser, verification arithmetic, settlement logic, UI state changes
- Simulated: network cell, device telemetry, settlement rate, partner responses

## Run locally
npm install
npm run dev

## Demo path
/wallet -> /operator -> optimise -> run -> verify -> settle -> /wallet

## Architecture
[short diagram / screenshot]

## Team
- Person 1: core engine / integration
- Person 2: operator experience
- Person 3: resident experience / UX / pitch

## Production path
DNSP topology + metering/aggregator interface + formal settlement arrangement + production M&V.
```

19. Questions the Implementation Must Survive

Question	Defensible answer
"Is Dinh receiving someone else's solar electricity?"	No. The prototype targets local flexible demand and allocates verified program value. It does not claim peer-to-peer electron routing.
"Is DAPTO-01 real?"	The hero electrical cell and constraint are simulated/approximate and labelled as such. Production needs DNSP topology/telemetry.

Question	Defensible answer
"Where does \$0.80/kWh come from?"	It is a simulated prototype settlement parameter. The production funding source requires an agreed retailer/aggregator/network program.
"Why 60% Equity Floor?"	Prototype governance parameter, not a universal scientific threshold. The technical contribution is that the rule is explicit, validated and auditable.
"What if the forecast is wrong?"	Below the confidence threshold the system does not dispatch/settle; a no-event state is valid.
"Why not just optimise the battery?"	Sunshine Wallet adds equity targeting, explainability, verification and settlement governance to network flexibility.
"What actually works today?"	The decision engine, simulation, M&V arithmetic, policy validation, allocation and wallet update. Physical/market integrations are mocked.

20. Final Priority Checklist

P0 - complete by Hour 11

- ☐ Shared types frozen
- ☐ Deterministic demoScenario fixture
- ☐ Dinh wallet initial state
- ☐ DAPTO-01 operator state
- ☐ 6 resources with correct units
- ☐ Hard eligibility checks
- ☐ Weighted optimiser
- ☐ Partial final resource dispatch
- ☐ Simulated event response
- ☐ Verification + confidence gate
- ☐ Settlement + Equity Floor validation
- ☐ Wallet update
- ☐ Reset Demo
- ☐ Complete clickable flow

P1 - before feature freeze if safe

- ☐ Polished before/after chart
- ☐ Data classification badges
- ☐ Invalid-policy feedback
- ☐ No-event / low-confidence state
- ☐ Three read-only area context cards

Do not build

- [] Native mobile app
- [] Real auth
- [] Live DNSP/retailer/battery integration
- [] Full GIS
- [] ML
- [] Microservices
- [] Production database if it threatens the flow

21. Minimum and Ideal Submission

Minimum successful submission

Two experiences are enough: operator DAPTO-01 event flow and Dinh wallet. The optimiser must genuinely select/reject resources; the event must genuinely change the interval dataset; verification, settlement and the wallet must genuinely recalculate. Everything external may be simulated and labelled.

Ideal submission within 25 hours

The minimum version plus polished charts, data-provenance badges, one invalid-policy demonstration, a no-event state, three read-only opportunity-area cards, a strong README and a rehearsed 3-5 minute demo.

One-sentence development philosophy

One codebase, one cell, one event, one resident, one auditable chain of cause and effect - finish that story before adding anything else.

22. Source Alignment

This plan preserves the concept brief's strongest design principles: FIND -> CATCH -> SHARE -> PROVE; Sunshine Cells rather than loose suburb/feeder assumptions; an explicit Equity Floor; verification against a counterfactual; resident simplicity; operator technical transparency; and honest labelling of simulated data. The implementation choices above are hackathon refinements, not claims that the production system can be built in 25 hours.

Source document: Sunshine Wallet - A Local Energy Equity Orchestrator for Wollongong, working concept and system design brief, 22 August 2026.